

Лекция 7: Трансформеры (1/2)

21 июня 2026 г.

Трансформеры доказали свою эффективность в качестве архитектуры DNN в самых разных областях.

- Поиск информации
- Машинный перевод
- Распознавание речи
- Преобразование текста в речь
- Компьютерное зрение
- Обучение с подкреплением
- Генеративный ИИ (LLM-модели, модели преобразования текста в изображение, подписывание изображений и т. д.)
- Многочисленные специализированные системы (например, AlphaFold)
- ...

В качестве основного примера использования мы будем рассматривать поиск/извлечение информации.

- Найдите мне все рейсы из Бостона в аэропорт Лос-Анджелеса завтра утром
- Сколько покупателей бросили свои корзины с покупками?
- Найдите все контракты, срок действия которых истекает в следующем месяце.

Сегодня мы сосредоточимся на этом примере, связанном с путешествиями

- Найдите мне все рейсы из Бостона в аэропорт Лос-Анджелеса завтра утром

Сегодня мы сосредоточимся на этом примере, связанном с путешествиями

- Найдите мне все рейсы из Бостона в аэропорт Лос-Анджелеса завтра утром
- В подобных сценариях использования обычно применяется следующий подход: преобразование запроса на естественном языке в структурированный запрос (т.е. SQL), который можно использовать для поиска информации в базе данных.

Сегодня мы сосредоточимся на этом примере, связанном с путешествиями

- Найдите мне все рейсы из Бостона в аэропорт Лос-Анджелеса завтра утром
- В подобных сценариях использования обычно применяется следующий подход: преобразование запроса на естественном языке в структурированный запрос (т.е. SQL), который можно использовать для поиска информации в базе данных.
- Для этого нам необходимо автоматически извлекать сущности, связанные с путешествиями, из запроса на естественном языке.

Мы будем использовать набор данных Информационной системы авиакомпаний (ATIS)¹.

¹<https://aclanthology.org/H90-1021/>

Извлечение «сущностей» из естественного языка

- Дана запрос на естественном языке...

Извлечение «сущностей» из естественного языка

- Дана запрос на естественном языке...
- Ввод: Я хочу вылететь из Бостона в 7 утра и прибыть в Денвер в 11 утра.

Извлечение «сущностей» из естественного языка

- Дана запрос на естественном языке...
- Ввод: Я хочу вылететь из Бостона в 7 утра и прибыть в Денвер в 11 утра.
- ... классифицировать каждое слово в запросе по соответствующему «слоту».

Классифицируйте каждое слово в запросе по соответствующему «слоту» — Пример

I want to fly from	00000
boston	B-fromloc.city_name
at	0
7 am	B-depart_time.time I-depart_time.time
and arrive in	000
denver	B-toloc.city_name
at	0
11	B-arrive_time.time
in the	00
morning	B-arrive_time.period_of_day

Типы слотов в наборе данных ATIS

B-aircraft_code,
B-airline_code,
B-airport_code,
B-arrive_date.date_relative,
B-arrive_date.day_name,
B-arrive_date.day_number,
B-arrive_date.month_name,
B-arrive_date.today_relative,
B-arrive_time.end_time,
B-arrive_time.period_mod,
B-arrive_time.period_of_day,
...
I-round_trip,
I-stoploc.city_name,
I-time, I-today_relative,

123 ВОЗМОЖНЫХ
СЛОТОВ!

Как можно решить эту задачу многоклассовой классификации слов и временных интервалов?

I want to fly from boston at 7 am and arrive in denver at 11 in the morning

O O O O O B-fromloc.city_name O B-depart_time.time
I-depart_time.time O O O B-toloc.city_name O B-arrive_time.time
O O B-arrive_time.period_of_day

Как можно решить эту задачу многоклассовой классификации слов и временных интервалов?

I want to fly from boston at 7 am and arrive in denver at 11 in the morning

O O O O O B-fromloc.city_name O B-depart_time.time
I-depart_time.time O O O B-toloc.city_name O B-arrive_time.time
O O B-arrive_time.period_of_day

Каждое из 18 слов, перечисленных выше, должно быть отнесено к одному из 123 типов слотов!

Если бы мы могли пропустить запрос через DNN и сгенерировать 18 выходных сигналов (по одному для каждого входного слова в запросе), мы могли бы прикрепить к каждому из этих 18 выходных сигналов функцию softmax с 123 параметрами.

Что нам необходимо учесть?

Мы хотим получить **выходные данные той же длины, что и входные** (чтобы мы могли классифицировать каждый выходной элемент по соответствующему типу слота).

Что нам необходимо учесть?

Мы хотим получить **выходные данные той же длины, что и входные** (чтобы мы могли классифицировать каждый выходной элемент по соответствующему типу слота).

Кроме того, мы хотели бы...

- Учитывайте **контекст** каждого слова.
- Учитывайте **порядок** слов.

Контекст имеет значение

Значение слова может кардинально меняться в зависимости от контекста.

Единого векторного вложения — например, GloVe — для всех контекстов, в которых может встречаться слово, недостаточно.

Контекст имеет значение

Значение слова может кардинально меняться в зависимости от контекста.

Единичного векторного волжения — например, GloVe — для всех контекстов, в которых может встречаться слово, недостаточно.

Слово	Примеры контекстов
see	I will see you soon I will see this project to its end I see what you mean
bank	I went to the bank to apply for a loan I am banking on the job offer coming through I am standing on the left bank
it	The animal didn't cross the street because it was too tired The animal didn't cross the street because it was too wide
station	The train left the station on time The radio station was playing 60s hits I was stationed on a remote island in Polynesia

Порядок имеет значение

добавьте свои собственные примеры

- Соответствует ВСЕМ требованиям, которые мы определили ранее
 - Учитывает контекст каждого слова
 - Учитывает порядок слов
 - Может генерировать выходные данные, имеющие ту же длину, что и входные

Трансформер архитектура

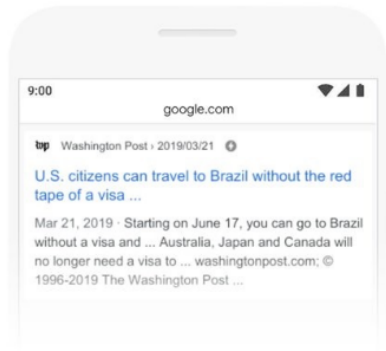
- Соответствует ВСЕМ требованиям, которые мы определили ранее
 - Учитывает контекст каждого слова
 - Учитывает порядок слов
 - Может генерировать выходные данные, имеющие ту же длину, что и входные
- Разработано в 2017 году. Оказывает значительное и продолжающееся влияние на глубокое обучение.

Влияние трансформатора на Google Поиск²



2019 brazil traveler to usa need a visa

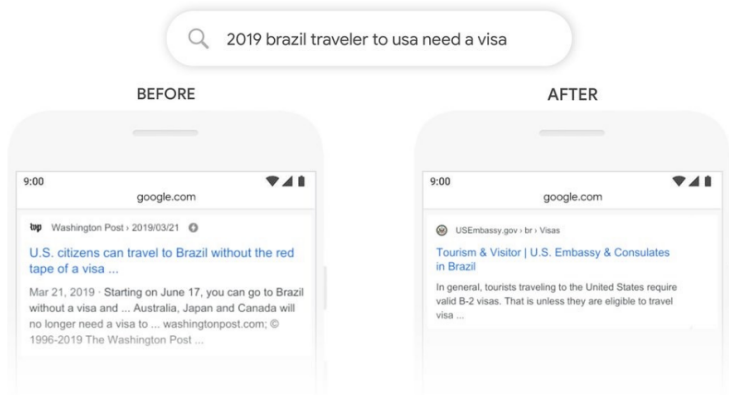
BEFORE



²[https:](https://blog.google/products/search/search-language-understanding-bert/)

[//blog.google/products/search/search-language-understanding-bert/](https://blog.google/products/search/search-language-understanding-bert/)

Влияние трансформатора на Google Поиск³



³<https://blog.google/products/search/search-language-understanding-bert/>

Трансформер архитектура⁴

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

<https://arxiv.org/abs/1706.03762>

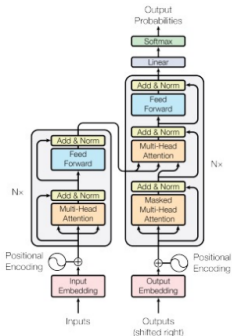


Figure 1: The Transformer - model architecture.

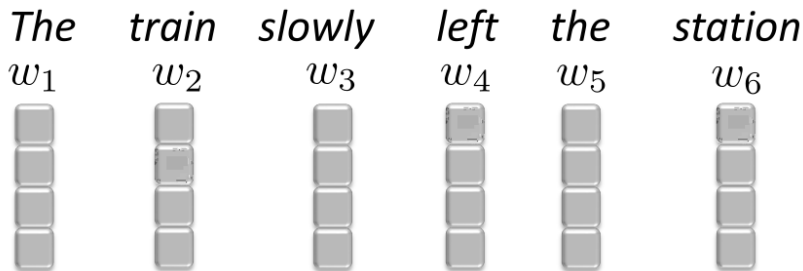
⁴<https://arxiv.org/abs/1706.03762>

Начнём с этого

- Как учитывать окружающий контекст каждого слова
- Как учитывать порядок слов
- Как сгенерировать выходные данные, имеющие ту же длину, что и входные данные

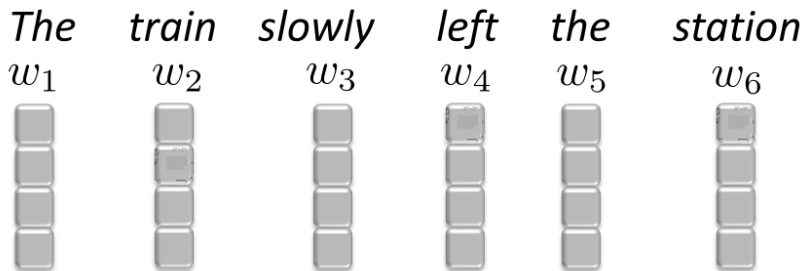
Как учитывать окружающий контекст каждого слова

От эмбедингов к контекстным эмбедингам



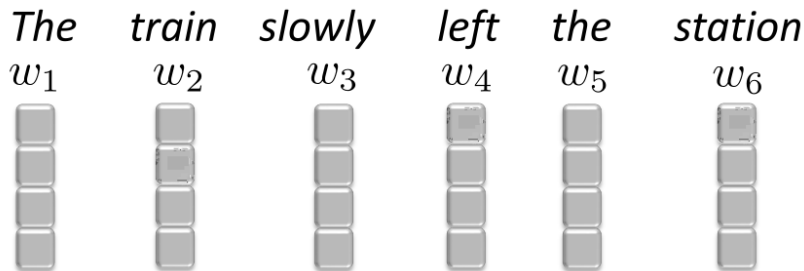
- Мы можем легко получить отдельные **stand-alone embeddings** для всех слов.

От эмбедингов к контекстным эмбедингам



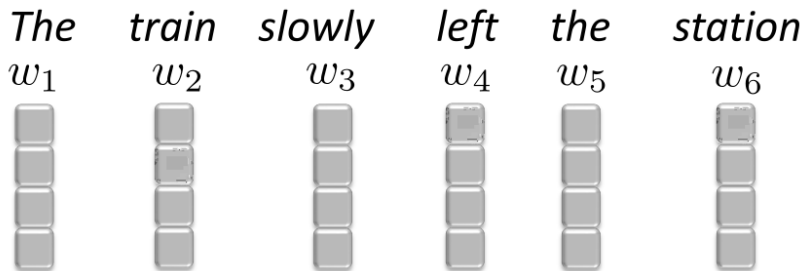
- Мы можем легко получить отдельные **stand-alone embeddings** для всех слов.
- Как можно изменить embedding слова «station», чтобы оно включало в себя остальные слова?

От эмбедингов к контекстным эмбедингам



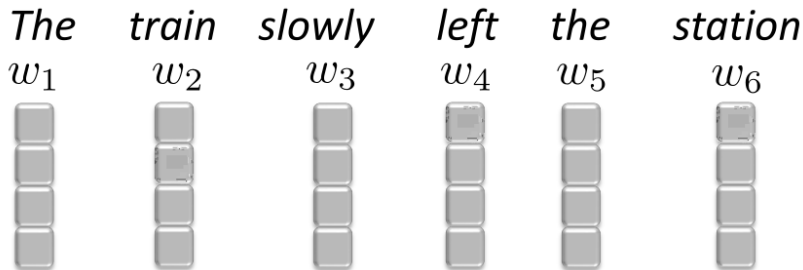
- Представьте, что мы каким-то образом знаем, сколько внимания следует уделять другим словам, то есть, какой вес им придавать.

От эмбедингов к контекстным эмбедингам



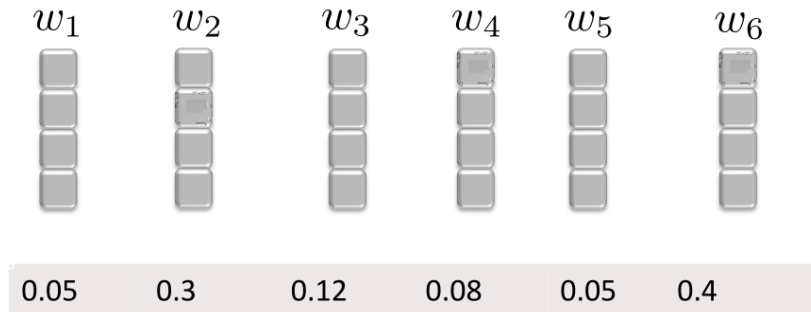
- Представьте, что мы каким-то образом знаем, сколько внимания следует уделять другим словам, то есть, какой вес им придавать.
- Интуитивно: Какие слова должны иметь наибольший вес, а какое слово имеет наименьший вес?

От эмбедингов к контекстным эмбедингам



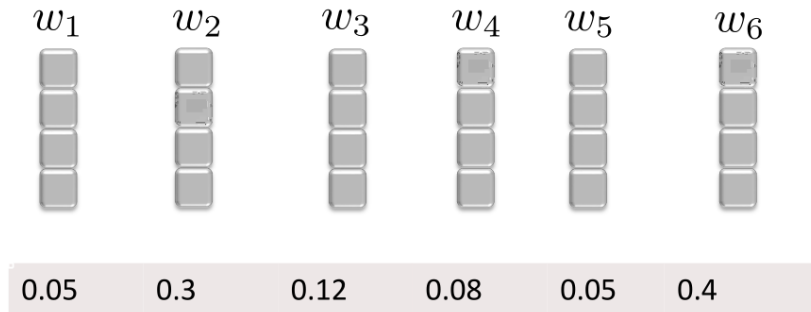
Следует уделять большое внимание слову «station», немного — словам «slowly» и «left», и совсем ничего — артиклю «the».

От эмбедингов к контекстным эмбедингам



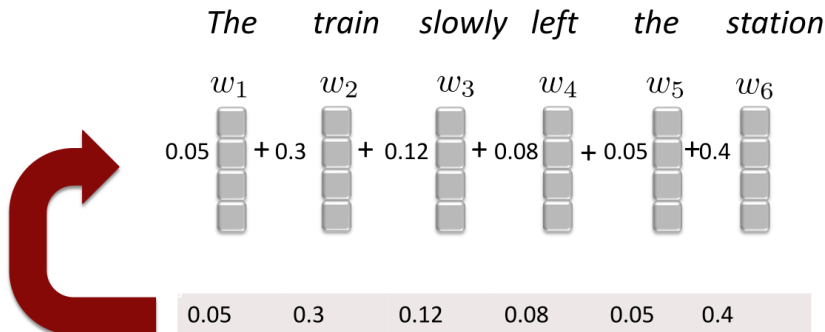
Может быть, что-то вроде этого?

От эмбеддингов к контекстным эмбеддингам



Как мы можем использовать эти веса для «контекстуализации» stand-alone embedding w_6 для «station»?

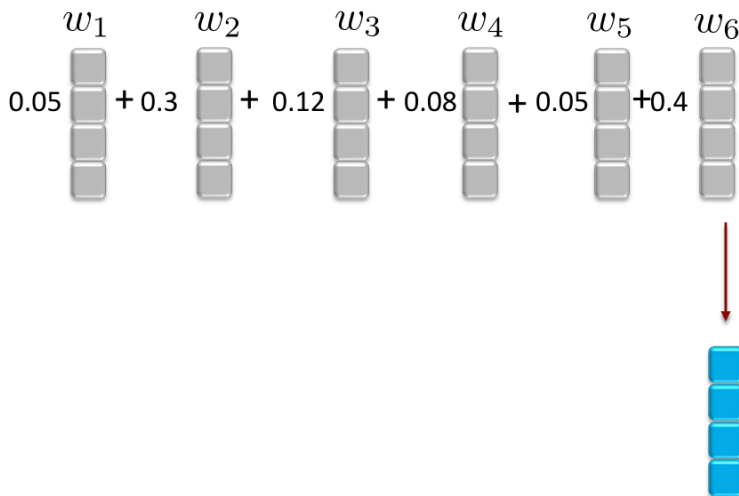
От эмбедингов к контекстным эмбедингам



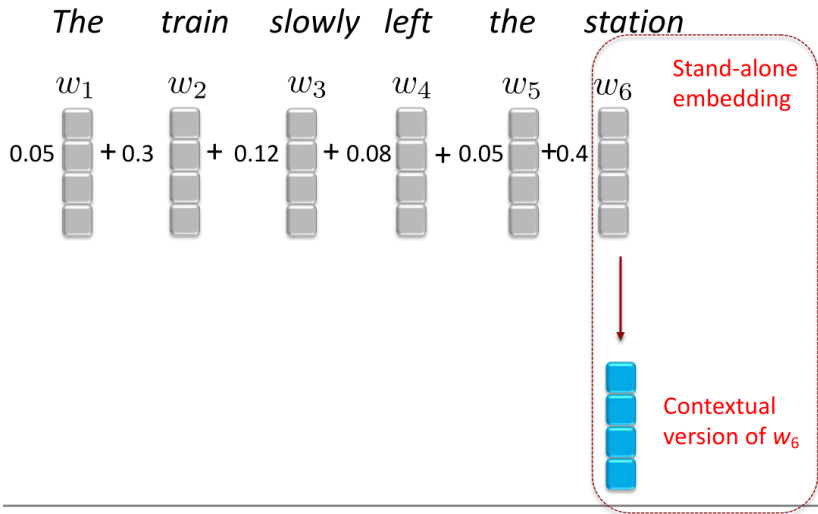
Идея: Для каждого слова мы можем вычислить взвешенное среднее stand-alone embeddings всех слов в предложении.

От эмбедингов к контекстным эмбедингам

The train slowly left the station

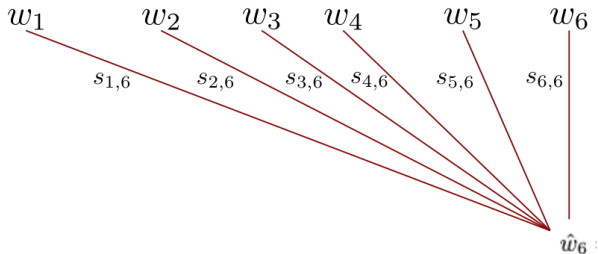


От эмбедингов к контекстным эмбедингам



Давайте напомним это более формально

The train slowly left the station



Stand-alone
embedding

$$\hat{w}_6 = s_{1,6}w_1 + s_{2,6}w_2 + s_{3,6}w_3 + s_{4,6}w_4 + s_{5,6}w_5 + s_{6,6}w_6$$

Contextual
version of w_6

Как следует выбирать весовые коэффициенты для заданного слова (например, «station»)?

Как следует выбирать весовые коэффициенты для заданного слова (например, «station»)?

Интуиция

- Вес слова должен быть пропорционален степени его связи со словом «station».

Как следует выбирать весовые коэффициенты для заданного слова (например, «station»)?

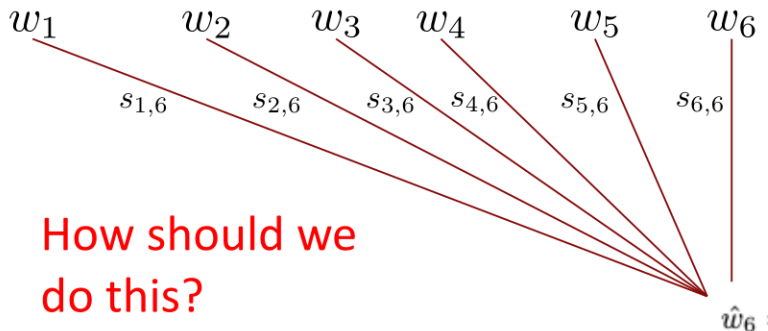
Интуиция

- Вес слова должен быть пропорционален степени его связи со словом «station».
- Один из способов количественной оценки степени «связанности» двух слов: скалярное произведение их stand-alone embeddings.

Как скалярные произведения измеряют «связанность»

Скалярные произведения между embeddings являются ключевым элементом, но нам нужно сделать еще кое-что, чтобы придать им правильный⁵ вес.

The train slowly left the station

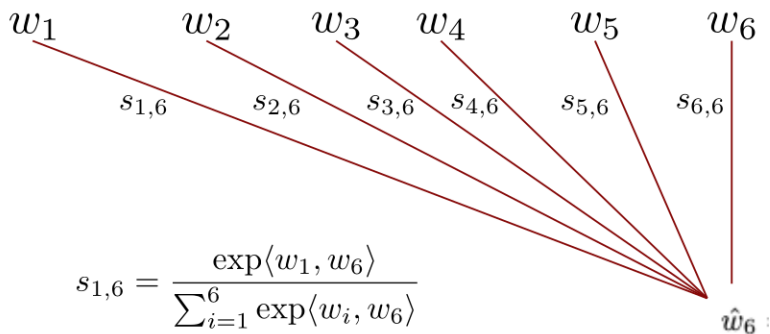


How should we
do this?

⁵ неотрицательные, и в сумме дают 1.0

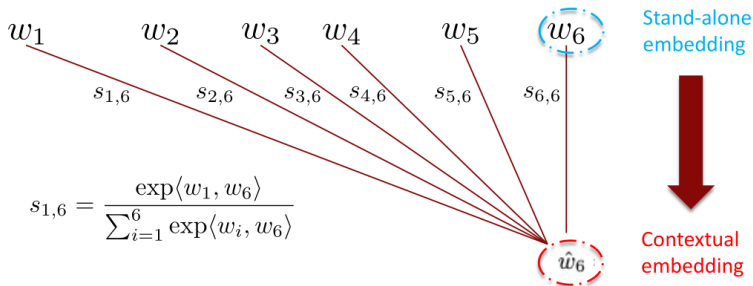
Поскольку скалярные произведения могут быть отрицательными, мы можем возвести их в степень, а затем нормализовать (помните функцию softmax?).

The train slowly left the station



Кратко: От эмбеддингов к контекстным эмбеддингам

The train slowly left the station



$$\hat{w}_6 = s_{1,6}w_1 + s_{2,6}w_2 + s_{3,6}w_3 + s_{4,6}w_4 + s_{5,6}w_5 + s_{6,6}w_6$$

Выбирая весовые коэффициенты таким образом, векторное представление слова приближается к векторным представлениям других слов в текущем контексте пропорционально степени их родства



- Слово «station» имеет множество значений.

Выбирая весовые коэффициенты таким образом, векторное представление слова приближается к векторным представлениям других слов в текущем контексте пропорционально степени их родства



- Слово «station» имеет множество значений.
 - В данном контексте слово «train» тесно связано со словом «station» и, следовательно, оказывает на него сильное «притяжение».

Выбирая весовые коэффициенты таким образом, векторное представление слова приближается к векторным представлениям других слов в текущем контексте пропорционально степени их родства



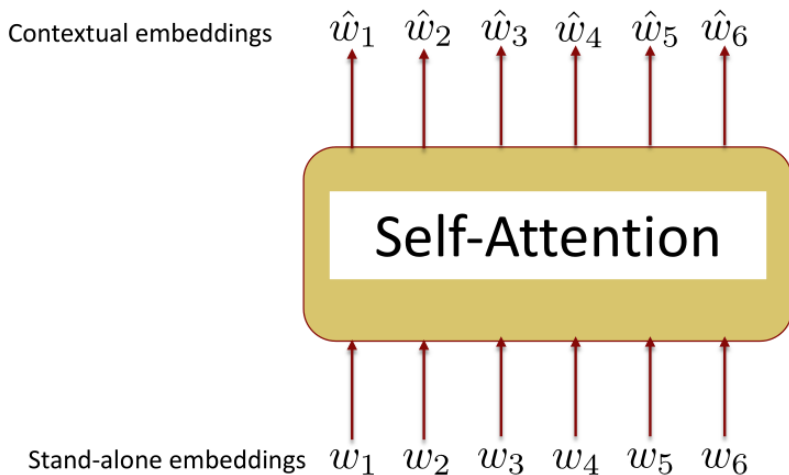
- Слово «station» имеет множество значений.
 - В данном контексте слово «train» тесно связано со словом «station» и, следовательно, оказывает на него сильное «притяжение».
 - Слово «radio» также связано со словом «station», но не встречается в текущем контексте, поэтому (автоматически) имеет нулевой вес.

Выбирая весовые коэффициенты таким образом, векторное представление слова приближается к векторным представлениям других слов в текущем контексте пропорционально степени их родства

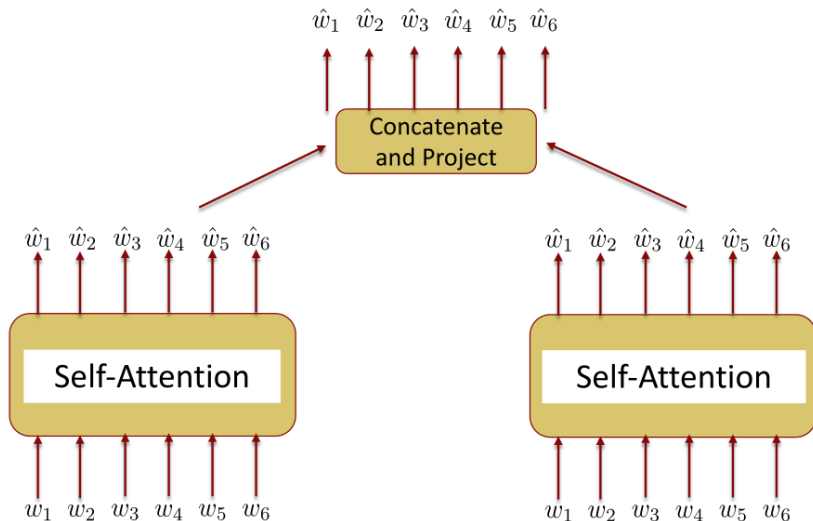


- Слово «station» имеет множество значений.
 - В данном контексте слово «train» тесно связано со словом «station» и, следовательно, оказывает на него сильное «притяжение».
 - Слово «radio» также связано со словом «station», но не встречается в текущем контексте, поэтому (автоматически) имеет нулевой вес.
- Перемещая station ближе к train (или, другими словами, уделяя больше «внимания» train), мы контекстуализируем расположение station в контексте train, platform, departure и т. д.

Эта операция называется слоем «самовнимания» и может быть выполнена очень эффективно с помощью матричных операций.



Ключевая настройка: Многоголовочное⁶ внимание(Multi-Head Attention)

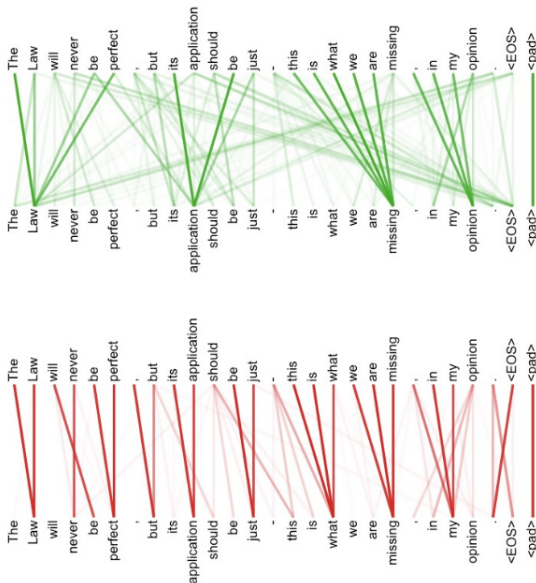


⁶Аналогично использованию нескольких фильтров в сверточном слое

Это помогает нам «обращать внимание» на многочисленные закономерности, которые могут присутствовать в одном предложении

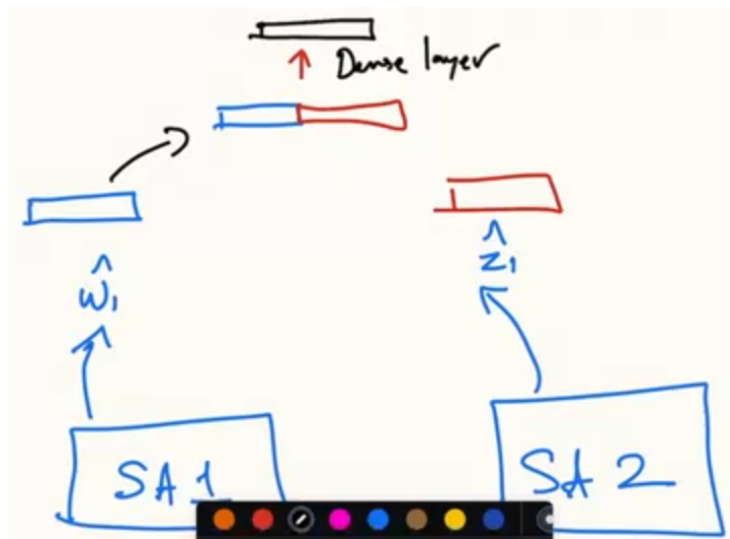
- Закономерности, связанные с 'временами'
- Шаблоны, связанные с 'стилями'
- Закономерности, связанные с взаимоотношениями между элементами предложения
- ...

Разные «головы внимания» обучаются разным шаблонам

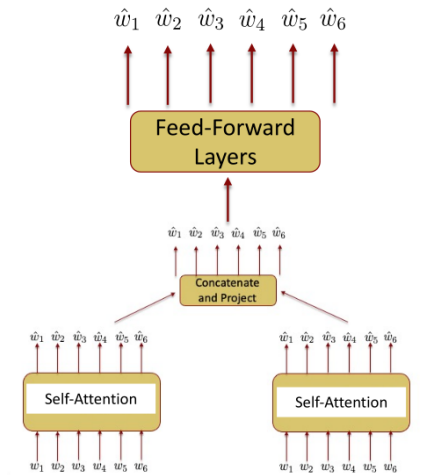


<https://arxiv.org/abs/1706.03762>

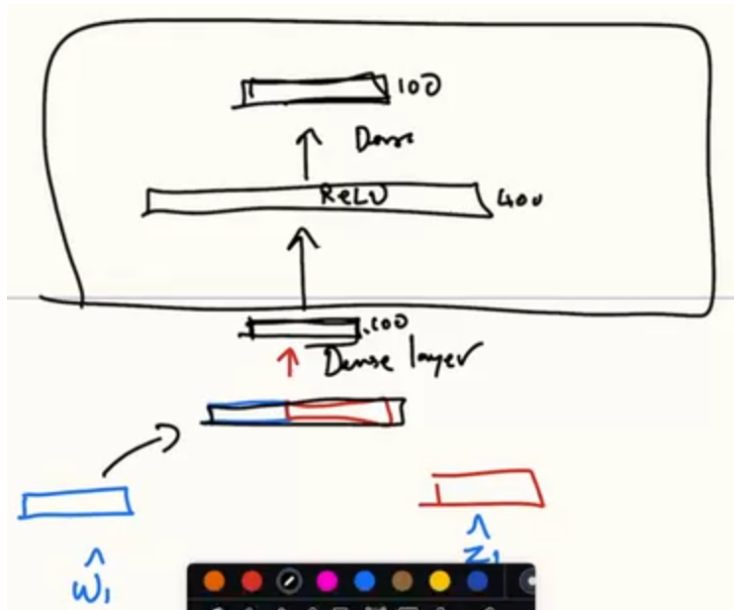
concatenate and project



Ключевое изменение: В конце добавляем некоторую нелинейность с помощью feed-forward слоев



feed forward

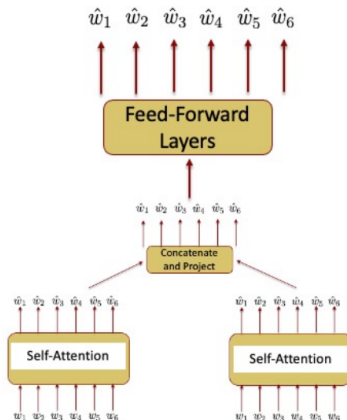


История на данный момент

End with contextual embeddings

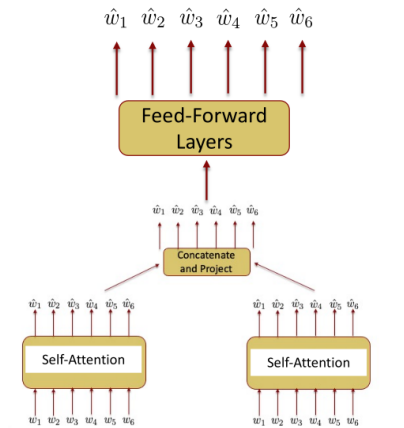


Start with random embeddings



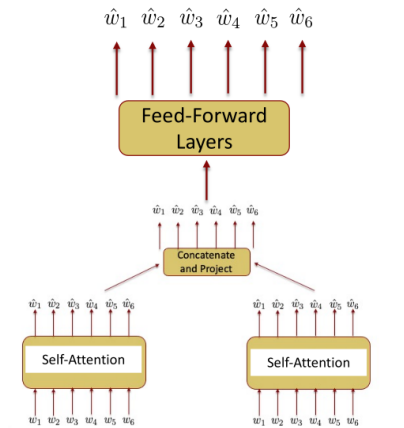
Мы выполнили 2 из 3 требований

- ✓ Учитывать контекст слов
- ? Учитывать порядок слов
- ✓ Генерировать выходные данные, имеющие ту же длину, что и входные



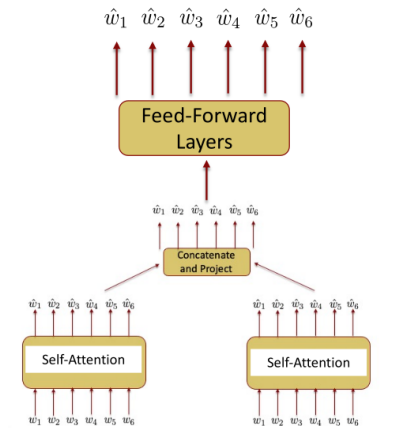
Учитывает ли данная архитектура порядок слов?

- ✓ Учитывать контекст слов
- ? Учитывать порядок слов
- ✓ Генерировать выходные данные, имеющие ту же длину, что и входные



Архитектура не учитывает порядок слов

Мы можем изменить порядок слов в предложении, и в итоге получим точно такие же контекстные вложенные представления.



Решение: Позиционное кодирование

- Добавьте позицию каждого слова в предложении к его stand-alone embedding.
- Наши input word embeddings слов будут представлять собой сумму двух вещей:
 - обычное 'stand-along' embedding +
 - positional embedding, которое обозначает позицию слова в предложении.

Позиционное кодирование — пример

Stand-alone embedding

Word	Dimension 1	Dimension 2
[UNK]	6.1	-3.2
cat	0.5	7.1
mat	-2	-3.1
I	0.1	3.4
sit	1.2	5.3
love	6.1	7.2
the	0.1	0.1
you	5.0	3.2
on	2.0	4.1

Position embedding

Position	Dimension 1	Dimension 2
0	1.3	3.9
1	6.3	3.7
2	0.6	8.1
3	-2.3	-4.1
4	0.14	5.4
5	1.29	3.3
6	6.12	-2.2
7	0.11	2.1
8	5.03	-5.2
9	2.05	-4.1

Позиционное кодирование — пример

Stand-alone embedding

Word		
[UNK]	6.1	-3.2
cat	0.5	7.1
mat	-2.0	-3.1
l	0.1	3.4
sat	1.2	5.3
love	6.1	7.2
the	0.1	0.1
you	5.0	3.2
on	2.0	4.1

Position embedding

Position		
0	1.3	3.9
1	6.3	3.7
2	0.6	8.1
3	-2.3	-4.1
4	0.14	5.4
5	1.29	3.3
6	6.12	-2.2
7	0.11	2.1
8	5.03	-5.2
9	2.05	-4.1

“cat sat mat”

cat	0.5	7.1
0	1.3	3.9
sum	1.8	11.0

Позиционное кодирование — пример

Stand-alone embedding

Word		
[UNK]	6.1	-3.2
cat	0.5	7.1
mat	-2.0	-3.1
l	0.1	3.4
sat	1.2	5.3
love	6.1	7.2
the	0.1	0.1
you	5.0	3.2
on	2.0	4.1

Position embedding

Position		
0	1.3	3.9
1	6.3	3.7
2	0.6	8.1
3	-2.3	-4.1
4	0.14	5.4
5	1.29	3.3
6	6.12	-2.2
7	0.11	2.1
8	5.03	-5.2
9	2.05	-4.1

“cat sat mat”

cat	0.5	7.1
0	1.3	3.9
sum	1.8	11.0

sat	1.2	5.3
1	6.3	3.7
sum	7.5	9.0

mat	-2.0	-3.1
2	0.6	8.1
sum	-1.4	5.0

Позиционное кодирование — пример

Stand-alone embedding

Word		
[UNK]	6.1	-3.2
cat	0.5	7.1
mat	-2.0	-3.1
I	0.1	3.4
sat	1.2	5.3
love	6.1	7.2
the	0.1	0.1
you	5.0	3.2
on	2.0	4.1

Position embedding

Position		
0	1.3	3.9
1	6.3	3.7
2	0.6	8.1
3	-2.3	-4.1
4	0.14	5.4
5	1.29	3.3
6	6.12	-2.2
7	0.11	2.1
8	5.03	-5.2
9	2.05	-4.1

“cat sat mat”

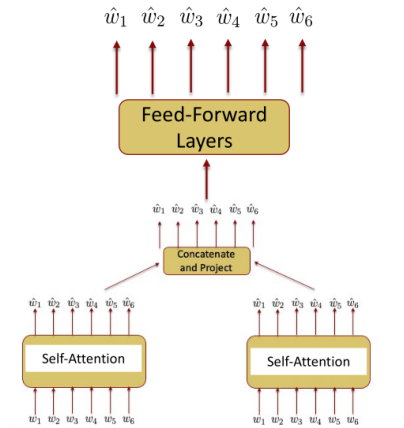
cat	0.5	7.1
0	1.3	3.9
sum	1.8	11.0

sat	1.2	5.3
1	6.3	3.7
sum	7.5	9.0

mat	-2.0	-3.1
2	0.6	8.1
sum	-1.4	5.0

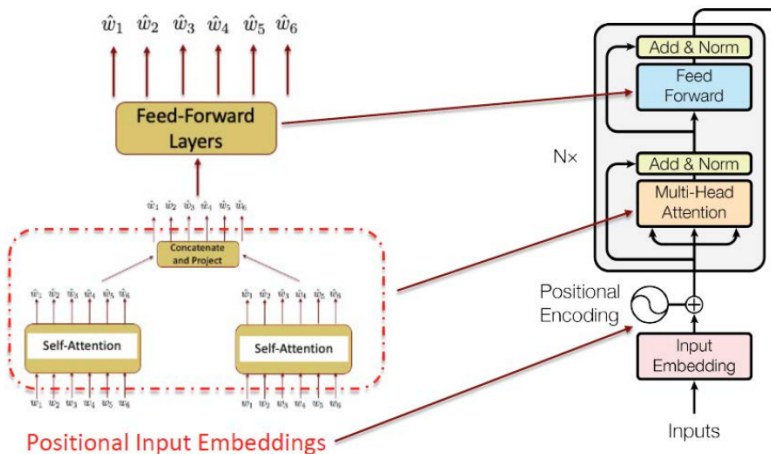
Мы выполнили все 3 требования

- ✓ Учитывать контекст слов
- ✓ Учитывать порядок слов
- ✓ Генерировать выходные данные, имеющие ту же длину, что и входные



Positional Input Embeddings

Это называется transformer encoder

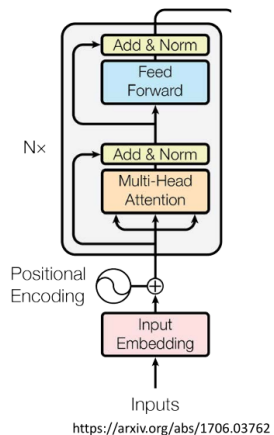


Positional Input Embeddings

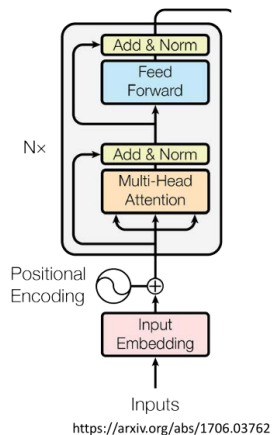
*with layernorm and residual connections (details in the next lecture)

<https://arxiv.org/abs/1706.03762>

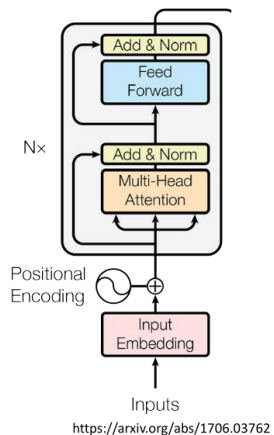
- Входные вложения могут представлять собой просто случайные вложения или предварительно обученные.



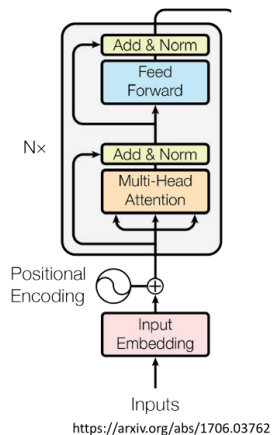
- Входные вложения могут представлять собой просто случайные вложения или предварительно обученные.
- Добавили позиционно-зависимое вложение, чтобы отобразить позицию каждого слова в предложении.



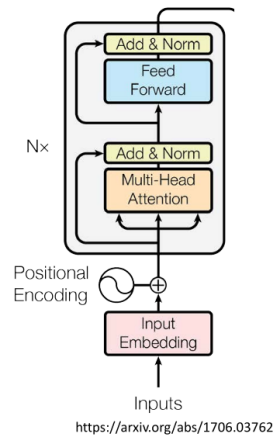
- Входные вложения могут представлять собой просто случайные вложения или предварительно обученные.
- Добавили позиционно-зависимое вложение, чтобы отобразить позицию каждого слова в предложении.
- Прохождение через механизм многоголового внимания позволяет получить контекстно-зависимое представление.



- Входные вложения могут представлять собой просто случайные вложения или предварительно обученные.
- Добавили позиционно-зависимое вложение, чтобы отобразить позицию каждого слова в предложении.
- Прохождение через механизм многоголового внимания позволяет получить контекстно-зависимое представление.
- Наконец, пропустите все это через простую feed-forward сеть.



- Входные вложения могут представлять собой просто случайные вложения или предварительно обученные.
- Добавили позиционно-зависимое вложение, чтобы отобразить позицию каждого слова в предложении.
- Прохождение через механизм многоголового внимания позволяет получить контекстно-зависимое представление.
- Наконец, пропустите все это через простую feed-forward сеть.
- Поскольку энкодеры имеют одинаковые по размеру входы и выходы, их можно соединять последовательно (т.е., располагать друг над другом) для увеличения возможностей моделирования.



Элементы transformer encoder, которые будут рассмотрены на следующей лекции.

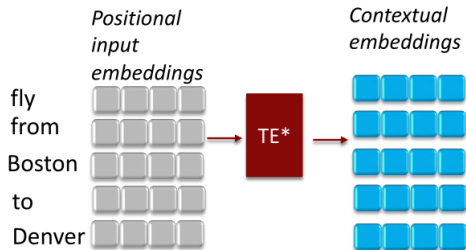
- Линейные проекции incoming embeddings в три различных пространства перед выполнением операции самовнимания.
- Остаточные соединения
- Нормализация слоев

Давайте применим transformer encoder к задаче преобразования слов в слоты!

Заполнение слотов трансформерами

fly	0
from	0
Boston	B-fromloc.city_name
to	0
Denver	B-to loc.city_name

Заполнение слотов трансформерами



O

O

B-fromloc.city_name

O

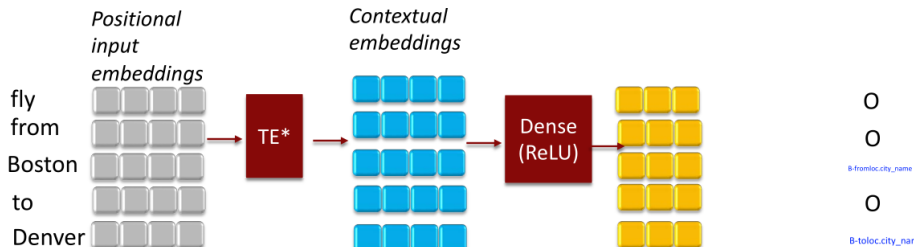
B-toloc.city_name

*Transformer Encoder



Indicate 4-element embedding vectors

Заполнение слотов трансформерами

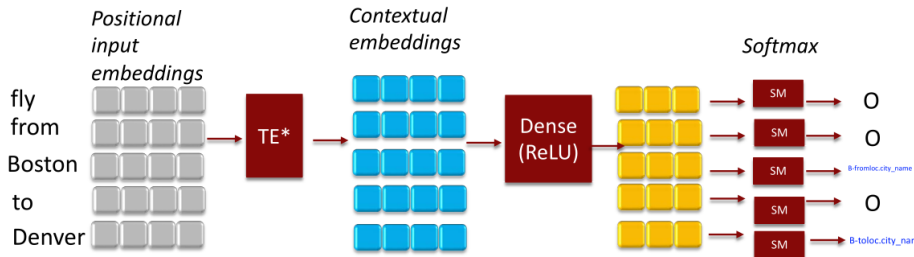


*Transformer Encoder



Indicate 4-element embedding vectors

Заполнение слотов трансформерами

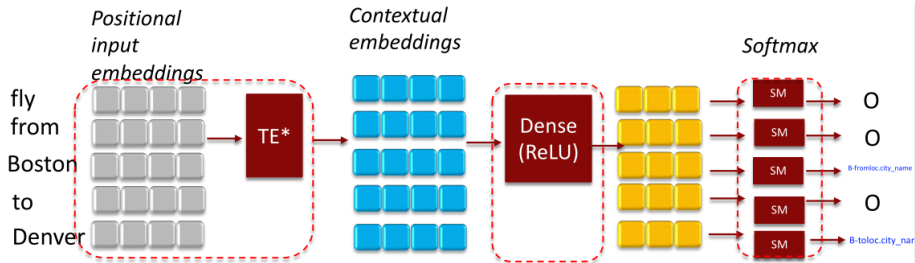


*Transformer Encoder



Indicate 4-element embedding vectors

Заполнение слотов трансформерами



 The weights in all these layers will get optimized by backprop

*Transformer Encoder



Indicate 4-element embedding vectors

[Ссылка](#) на решение.